

BÁO CÁO BÀI TẬP 5: KIỂM THỬ HIỆU NĂNG ỨNG DỤNG WEB E-COMMERCE (HW05-AI)

Môn học: Kiểm thử phần mềm (Software Testing)

Trường: Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM (HCMUS)

Mã bài tập: HW05 – Performance Testing (Exercise ID: HW05-AI)

Sinh viên thực hiện: BaoBeiii – MSSV: 23127327

Email sinh viên: 23127327@student.hcmus.edu.vn

Môi trường đo đạc: Máy trạm DESKTOP-BEIZU (Windows 11, Node.js v24.11.0, k6 v2.1.0)

Chiến lược kiểm thử: AI-Assisted Workflow & Rigorous Human Critique (AI-First kết hợp Phản biện Con người sâu sắc)

[!!IMPORTANT]

📺 LIÊN KẾT VIDEO DEMO BÁO CÁO (YOUTUBE DEMO LINKS)

- 🎬 Video Demo Task 1 (Đo tải k6, Bẫy 7 bug SUT, Phản biện AI & HTML Dashboard): <https://youtu.be/leNk4TxJ1D4>
- 🤖 Video Demo Agent Skill (Đóng gói Agent Skill Antigravity, CLI Metrics Gate & CI/CD): <https://youtu.be/vSyRLXkW-7k>

🌟 MỤC LỤC & DANH MỤC BÁO CÁO BÀN GIAO

- Tổng Quan Dự Án & Mục Tiêu Kỹ Thuật
- Cây Thư Mục Toàn Diện Của Repository
- Hướng Dẫn Khởi Chạy & Tái Hiện Kết Quả (Quick Start)
- Bảng Tổng Hợp Kết Quả Đo Tải Thực Nghiệm (Task 1)
- Tổng Hợp 7 Lỗi Hệ Thống SUT Được Cài Bẫy Bắt Trọn (BUG-01 đến BUG-07)
- Phản Biện Lỗi Suy Diễn AI & 4 Giải Pháp Tối Ưu Kiến Trúc (Task 2)
- Tự Động Hóa CI/CD GitHub Actions & Đóng Gói Agent Skill (Task 3)
- Các Báo Cáo Chuyên Sâu Đi Kèm

1. TỔNG QUAN DỰ ÁN & MỤC TIÊU KỸ THUẬT

Dự án thực hiện kiểm thử tải toàn diện cho hệ thống **EShop SUT** (Hệ thống thương mại điện tử Node.js Express + SQLite3) theo chu trình người dùng hoàn chỉnh (E2E User Journey):
\$\$\text{Login} \rightarrow \text{Browse Products} \rightarrow \text{View Detail} \rightarrow \text{Apply Coupon} \rightarrow \text{Checkout} \rightarrow \text{Record Coupon Usage} \rightarrow \text{View Order History}\$\$

Các Mục Tiêu Kỹ Thuật Đã Đạt Được:

- Đo lường định lượng chính xác 4 kịch bản tải: Load Test (Peak 50 VUs), Stress Test (Stepped 200 VUs), Spike Test (Flash Sale 150 VUs), Endurance / Soak Test (30 VUs trong 15 phút).
- Xác định ngưỡng phần cứng thực nghiệm (Empirical Hardware Thresholds):
 - Điểm gãy (Breaking Point): Nằm tại mức 120 - 150 VUs do hiện tượng nghẽn khóa ghi tập tin của SQLite (`SQLITE_BUSY`).
 - Thời gian hồi phục (Recovery Time): Đạt 14 - 16 giây sau khi kết thúc sốc tải 150 VUs về 5 VUs.
 - Đánh giá rò rỉ bộ nhớ (Memory Leak): Đạt chuẩn an toàn (Working Set RAM drift chỉ $+10.7 \text{ MB}$ sau 32 phút chịu tải liên tục với 576 mẫu đo telemetry).
- Bắt trọn 7 lỗi hệ thống của SUT bằng các kiểm tra chuyên sâu (Deep Assertions) trên k6.
- Tự động hóa CI/CD: Xây dựng workflow GitHub Actions tự động kiểm soát cổng chất lượng SLA và đóng gói Agent Skill tái sử dụng theo chuẩn Antigravity.

2. CÂY THƯ MỤC TOÀN DIỆN CỦA REPOSITORY

```
KTPM-HW05/
├── .agents/
│   ├── skills/
│   │   ├── performance-testing/
│   │   │   ├── SKILL.md                    # Đặc tả năng lực Agent Skill & Hướ
│   │   │   ├── scripts/
│   │   │   │   ├── extract_metrics.js      # CLI tool tự động phân tích p95 &
│   │   │   │   ├── references/
│   │   │   │   │   ├── sla_matrix.json    # Ma trận ngưỡng SLA chuẩn cho từng
│   │   │   │   │   └── examples/
│   │   │   │   │       └── sample_pr_comment.md # Mẫu comment tự động gửi vào Pull
│   ├── .github/
│   │   ├── workflows/
│   │   │   ├── performance-regression.yml # GitHub Actions CI/CD phát hiện hõ
│   ├── data/
│   │   ├── users.csv                      # 52 tài khoản test (admin, user1..
│   │   ├── products.csv                  # 15 sản phẩm đa dạng dải giá
│   │   └── orders.csv                    # 20 bản ghi đơn hàng mẫu và coupon
│   ├── evidence/
│   │   ├── load_test_metrics.txt          # Bảng chứng định lượng bài Load Te
│   │   ├── stress_breaking_point.txt      # Bảng chứng điểm gãy bài Stress Te
│   │   ├── spike_recovery_evidence.txt    # Bảng chứng thời gian hồi phục bài
│   │   ├── endurance_hardware_threshold.txt # Bảng chứng độ bền & ngưỡng phần c
│   │   └── resource_monitor_log.csv       # 576 mẫu đo liên tục CPU & RAM của
│   ├── report.md                         # BÁO CÁO CHÍNH TOÀN DIỆN (MAIN REP
│   ├── report.pdf                        # Bản PDF báo cáo chính thức (A4, T
│   ├── AI_Audit_Report.md                # Nhật ký kiểm toán AI minh bạch 6
│   ├── AI_Audit_Report.pdf               # Bản PDF nhật ký kiểm toán AI
│   ├── bug_reports.md                   # Mô tả chi tiết 7 lỗi SUT phát hiệ
│   ├── bug_reports.pdf                  # Bản PDF báo cáo 7 lỗi SUT
│   ├── Human_Review_Report.md            # Báo cáo phản biện 6 lỗi kịch bản
│   ├── Human_Review_Report.pdf           # Bản PDF phản biện kịch bản AI Tas
│   ├── test_cases.md                    # Ma trận và đặc tả toàn bộ ca kiểm
│   ├── test_cases.pdf                   # Bản PDF ma trận ca kiểm thử
│   ├── Video_Demo_Script.md              # Kịch bản chi tiết quay video demo
│   ├── Video_Demo_Script.pdf             # Bản PDF kịch bản video demo
│   ├── README.pdf                       # Bản PDF hướng dẫn dự án & Bảng tự
│   ├── git_commit_log.txt                # Trích lục toàn bộ lịch sử Git com
│   ├── results/
│   │   ├── load/
│   │   │   ├── summary.html              # Dashboard HTML tương tác kết quả
│   │   │   ├── metrics.json              # Dữ liệu JSON phân tích trễ p90, p
│   │   │   ├── raw_metrics.csv           # Mẫu đo chi tiết từng request
│   │   │   └── raw_load.jtl              # File log tương thích listener JMe
│   │   ├── stress/
│   │   │   └── summary.html               # Dashboard HTML tương tác kết quả
```

├── metrics.json	# Dữ liệu JSON phân tích trễ p90, p
├── raw_metrics.csv	# Mẫu đo chi tiết từng request
└── raw_stress.jtl	# File log tương thích listener JMe
├── spike/	
│ ├── summary.html	# Dashboard HTML tương tác kết quả
│ ├── metrics.json	# Dữ liệu JSON phân tích trễ p90, p
│ ├── raw_metrics.csv	# Mẫu đo chi tiết từng request
│ └── raw_spike.jtl	# File log tương thích listener JMe
└── endurance/	
│ ├── summary.html	# Dashboard HTML tương tác kết quả
│ ├── metrics.json	# Dữ liệu JSON phân tích trễ p90, p
│ ├── raw_metrics.csv	# Mẫu đo chi tiết từng request
│ └── raw_endurance.jtl	# File log tương thích listener JMe
├── scripts/	
│ ├── 23127327_Load_20260902.js	# Kịch bản k6 Load Test (Peak 50 VU
│ ├── 23127327_Load_20260902.jmx	# Kịch bản JMeter Load Test (View R
│ ├── 23127327_Stress_20260902.js	# Kịch bản k6 Stress Test (Stepped
│ ├── 23127327_Stress_20260902.jmx	# Kịch bản JMeter Stress Test (Summ
│ ├── 23127327_Spike_20260902.js	# Kịch bản k6 Spike Test (Flash Sal
│ ├── 23127327_Spike_20260902.jmx	# Kịch bản JMeter Spike Test (Aggre
│ ├── 23127327_Endurance_20260902.js	# Kịch bản k6 Endurance Test (30 VU
│ ├── k6-reporter.js	# Thư viện offline tạo HTML Dashboa
│ ├── k6-summary.js	# Thư viện offline format text summ
│ ├── papaparse.js	# Thư viện offline nạp CSV tham số
│ ├── extract_metrics.js	# CLI tool trích xuất metrics & SLA
│ ├── convert_to_jtl.js	# Tiện ích chuyển đổi metrics sang
│ ├── monitor_resources.ps1	# Script PowerShell giám sát CPU/RA
│ ├── seed_test_users.js	# Tiện ích nạp tự động 50 users tes
│ ├── reset_lockout.js	# Tiện ích mở khóa tài khoản dính F
│ └── reset_database.js	# Tiện ích khôi phục toàn diện CSDL
├── .gitignore	# Cô lập repo nộp bài khỏi eshop-su
└── README.md	# Tài liệu tổng quan toàn bộ đồ án

3. HƯỚNG DẪN KHỞI CHẠY & TÁI HIỆN KẾT QUẢ (QUICK START)

3.1 Yêu Cầu Môi Trường (Prerequisites)

- **Node.js:** v18+ (Đã kiểm thử trên v24.11.0 và v20 LTS).
- **k6:** v0.40+ (Đã kiểm thử trên Grafana k6 v2.1.0 tại `C:\Program Files\k6\k6.exe`).
- **PowerShell:** Chạy trên Windows hoặc Terminal trên Linux/macOS.

3.2 Khởi Động SUT Backend

```
# 1. Di chuyển vào thư mục backend của SUT
cd eshop-sut/backend

# 2. Cài đặt các gói phụ thuộc
npm install

# 3. Khởi chạy máy chủ Express trên cổng 3000
node server.js
```

Kiểm tra sức khỏe: Mở trình duyệt truy cập `http://localhost:3000/api/products` nhận về mảng JSON sản phẩm.

3.3 Nạp Dữ Liệu Test & Reset Trạng Thái Sạch

```
# Nạp 50 user test và reset logout
node scripts/reset_lockout.js
node scripts/seed_test_users.js
```

3.4 Bật Giám Sát Tài Nguyên Hệ Thống

Mở một cửa sổ PowerShell mới và chạy:

```
powershell -ExecutionPolicy Bypass -File scripts\monitor_resources.ps1
```

Script sẽ tự động tìm đúng PID của `node.exe` đang lắng nghe port 3000 và ghi log CPU %, Working Set RAM mỗi 3 giây vào `evidence/resource_monitor_log.csv`.

3.5 Thực Thi 4 Bài Kiểm Thử Bằng k6

```
# 1. Chạy Load Test (Peak 50 VUs - 8 phút)
k6 run --out csv=results/load/raw_metrics.csv scripts/23127327_Load_20260902.js

# 2. Chạy Stress Test (Bậc thang 10..200 VUs - ~4 phút)
k6 run --out csv=results/stress/raw_metrics.csv scripts/23127327_Stress_20260902.js

# 3. Chạy Spike Test (Đột biến 150 VUs trong 10s - ~2 phút)
k6 run --out csv=results/spike/raw_metrics.csv scripts/23127327_Spike_20260902.js

# 4. Chạy Endurance Test (Duy trì 30 VUs trong 15 phút)
k6 run --out csv=results/endurance/raw_metrics.csv scripts/23127327_Endurance_20260902.js
```

Sau khi chạy, mở trực tiếp các file `summary.html` trong `results/{scenario}/` bằng trình duyệt để xem Dashboard đồ họa tương tác!

3.6 Chạy Công Cụ Kiểm Tra Hồi Quy Hiệu Năng & SLA Gate

```
node scripts/extract_metrics.js results/load/metrics.json
```

3.7 Kiểm Tra Tính Toàn Vẹn Báo Cáo (Zero-Omission Integrity Linter)

```
# Tự động quét đối chiếu 1:1, bảo đảm không có bất kỳ lỗi hay mục nào bị bỏ sót
node scripts/verify_report_integrity.js
```

4. BẢNG TỔNG HỢP KẾT QUẢ ĐO TẢI THỰC NGHIỆM (TASK 1)

Thông Số Đo Đặc Thực Tế	Load Test (Tải Định)	Stress Test (Điểm Gãy)	Spike Test (Đột Biến)	Endurance Test (Độ Bền)
Cấu hình Virtual Users (VUs)	Peak 50 VUs	Bậc thang 10 $\rightarrow$ 200 VUs	Đột biến 150 VUs (10s)	Duy trì 30 VUs liên tục
Thời lượng thực thi	8m13.6s	4m19.5s	1m54.5s	15m03.4s
Tổng số Iterations hoàn tất	1.420	3.058	910	2.814
Tổng số HTTP Requests	5.235	17.279	5.068	15.736
Thông lượng trung bình (Throughput)	10.61 req/s	66.59 req/s (đỉnh ~135)	44.27 req/s	17.42 req/s
Dung lượng truyền nhận	3.1 MB / 869 kB	24 MB / 4.9 MB	12 MB / 1.4 MB	66 MB / 4.5 MB
Thời gian trễ trung bình (Mean)	1.16 ms	1.70 ms	1.75 ms	1.91 ms
Trễ phân vị đuôi \$p95\$ toàn kịch bản	1.74 ms (SLA < 1500ms)	4.93 ms	5.02 ms	5.73 ms
Trễ \$p95\$ Login / Checkout	1.84 ms / 6.95 ms	2.04 ms / 6.20 ms	2.28 ms / 6.29 ms	1.80 ms / 6.30 ms
Tỷ lệ lỗi (\$Error\text{Rate}\$)	42.8% (bẫy negative coupon)	10.72% (dưới tải 200 VU)	11.99% (dưới sốc tải)	11.70% (ổn định)
Số lỗi SUT bắt được	1.084 BUG-03, 6 BUG-01	2.329 BUG-03, 53 BUG-01	686 BUG-03	Bắt liên tục BUG-03
Hiện tượng & Đánh giá kỹ thuật	Đạt chuẩn SLA toàn diện	Điểm gãy: 120 - 150 VUs	Hồi phục sau: ~14 - 16s	Không rò rỉ RAM (drift +10.7MB)

5. TỔNG HỢP 7 LỖI HỆ THỐNG SUT ĐƯỢC CÀI BẮY BẮT TRỌN

Chi tiết xem tại tài liệu chuyên khảo: [bug_reports.md](#).

Mã Lỗi	Tên Lỗi & Vị Trí Code	Hành Vi Sai Lệnh Nghiêm Trọng	Bẫy Assertions & Số Lần Bắt
BUG-01	Sai công thức tính mã giảm giá phần trăm server.js:398	Áp dụng <code>total_amount * (1 - coupon.discount_value)</code> khiến số tiền giảm bị âm và tổng thanh toán bị đội giá gấp 10 lần.	Bắt 59 lần trong các bài đo tải (<code>total_discount >= 0</code>).
BUG-02	Lỗi hỏng SQL Injection tại tìm kiếm server.js:144	Ghép chuỗi trực tiếp <code>\${search}</code> vào câu lệnh SQL thay vì dùng Prepared Statement <code>?</code> .	Bẫy chuỗi <code>test' OR '1'='1</code> vạch trần lỗ hỏng SQLi.
BUG-03	Ép kiểu giá thành chuỗi ở ID chặn server.js:162	Ép kiểu <code>String(product.price)</code> nếu <code>id % 2 === 0</code> , phá vỡ hợp đồng API JSON.	Bắt 4.099 lần (<code>typeof price === 'number'</code>).
BUG-04	Khóa tài khoản sai logic FR-02 server.js:54-62	Tăng số lần thử sai <code>+2</code> thay vì <code>+1</code> và khóa tới 3 phút thay vì 30 giây.	Bẫy login mật khẩu sai và kiểm tra phản hồi 403.
BUG-05	Checkout không tính lại giỏ hàng server.js:296	Lấy trực tiếp <code>total_amount</code> từ client gửi lên mà không xác minh lại với CSDL, cho phép khách hàng sửa giá về 0đ.	Bẫy gian lận giá đơn hàng phát hiện lỗi hỏng logic.
BUG-06	Chuyển trạng thái đơn hàng phi lý server.js:550	Cho phép cập nhật đơn hàng đã bị hủy (<code>canceled</code>) chuyển thẳng thành đã giao (<code>delivered</code>).	Bẫy cập nhật trạng thái đơn hàng phát hiện sai luồng.
BUG-07	Lỗi hỏng Race Condition khi dùng mã server.js:425	Không bọc Transaction hoặc mutex lock, cho phép 1 user dùng vượt quá <code>max_uses_per_user</code> khi gửi đồng thời.	Bắt trong bài Stress Test 200 VUs.

6. PHẢN BIỆN LỖI SUY DIỄN AI & 4 GIẢI PHÁP TỐI ƯU KIẾN TRÚC (TASK 2)

Chi tiết phân tích mã nguồn và các khối so sánh Before/After xem trực tiếp tại: [report.md](#) (Mục 6).

6.1 Bắt 5 Lỗi Suy Diễn Ngụy Tạo Của AI Khi Đọc Log

- Ngụy biện giá trị trung bình (The Mean Fallacy):** AI khen nức nở Average Latency ~1.16ms mà bỏ qua phân vị trôi dạt ở bước ghi Checkout bị chậm gấp 6 lần bước đọc.
- Ngộ nhận tỷ lệ lỗi Load Test 42.8% là "server quá tải bị sập":** Thực tế là do kịch bản cố tình gửi các ca kiểm thử negative coupon để test logic nghiệp vụ (trả về HTTP 400 đúng thiết kế), hoàn toàn không phải server crash.

3. **Hiểu sai mã lỗi 403 Forbidden & Bỏ sót 7 Bug SUT:** Mã 403 là do cơ chế bảo mật tự động khóa tài khoản FR-02 hoạt động; AI bỏ sót hoàn toàn hơn 3.400 lỗi sai kiểu dữ liệu BUG-03 và lỗi tính giảm giá âm BUG-01.
4. **Chẩn đoán sai "Rò rỉ bộ nhớ" (Memory Leak False Positive):** AI chỉ thấy RAM tăng từ 59MB lên 96MB rồi phán rò rỉ; thực tế 576 mẫu đo chứng minh V8 Garbage Collection đã thu hồi RAM về 70MB sau tải (drift chỉ +10.7MB sau 32 phút, hệ thống an toàn tuyệt đối).
5. **Vạch trần lời khuyên sáo rỗng:** AI khuyên "nâng cấp phần cứng, dùng Kubernetes, viết lại bằng Rust/Go" mà không hiểu rằng điểm nghẽn cốt lõi nằm ở cơ chế khóa tập tin độc quyền của SQLite.

6.2 Bốn Đề Xuất Tối Ưu Hóa Kiến Trúc Dựa Trên Bằng Chứng Thực Nghiệm

1. **Kích hoạt SQLite WAL Mode & Cấu hình Busy Timeout:** Cấu hình `PRAGMA journal_mode = WAL;` và `PRAGMA busy_timeout = 5000;` để Đọc và Ghi diễn ra đồng thời không chặn nhau, giải quyết dứt điểm điểm gãy 120-150 VUs.
2. **Đánh chỉ mục (Indexing) các cột tìm kiếm và khóa ngoại:** Tạo B-Tree Indexes trên `products(name)`, `orders(user_id)`, `coupon_usage(coupon_id, user_id)` để giảm độ phức tạp từ $O(N)$ xuống $O(\log N)$.
3. **Sửa lỗi công thức BUG-01 & Giao dịch nguyên tử (Atomic Transactions):** Sửa công thức tính giảm giá phần trăm và bọc `db.serialize()` với `BEGIN IMMEDIATE TRANSACTION` triệt tiêu hoàn toàn Race condition BUG-07.
4. **Bộ nhớ đệm tầng ứng dụng (In-Memory / Redis Caching):** Cache in-memory danh mục sản phẩm (`GET /api/products`) với TTL 60 giây, giảm 80% tải đọc trực tiếp vào SQLite và giữ trễ đọc $< 0.5\text{ms}$.

7. TỰ ĐỘNG HÓA CI/CD GITHUB ACTIONS & ĐÓNG GÓI AGENT SKILL (TASK 3)

7.1 Workflow CI/CD GitHub Actions

(`.github/workflows/performance-regression.yml`)

- Tự động chạy khi có `push`, `pull_request` hoặc `workflow_dispatch`.
- Tự động dựng môi trường Node.js, clone SUT, khởi chạy backend và vòng lặp thăm dò sức khỏe bằng `curl`.
- Tự động chạy k6 regression test và kiểm tra cổng chất lượng SLA (SLA Quality Gate); tự động đánh rớt build nếu độ trễ thoái hóa vượt ngưỡng dung sai 20%.

7.2 Đóng Gói Trọn Bộ Agent Skill Tái Sử Dụng (`.agents/skills/performance-testing/`)

- Tuân thủ đặc tả Agentic Testing chuẩn Antigravity với `SKILL.md` chứa YAML metadata.
- Đi kèm CLI tool `scripts/extract_metrics.js` phân tích độ trễ \$p95\$, ma trận SLA `references/sla_matrix.json`, và mẫu comment PR `examples/sample_pr_comment.md`.

8. CÁC BÁO CÁO BÀN GIAO CHÍNH THỨC

1. 🏆 [report.md](#) (kèm bản in [report.pdf](#)): **BÁO CÁO CHÍNH TOÀN DIỆN (MAIN REPORT)** hợp nhất 10 mục của toàn bộ đề án theo Mục 14 đề bài (bao gồm toàn bộ phân tích AI Critique Task 2).
2. [AI_Audit_Report.md](#) (kèm bản in [AI_Audit_Report.pdf](#)): Nhật ký kiểm toán tương tác AI minh bạch 6 phiên làm việc với 4-5 điểm con người bắt sửa cho mỗi phần.
3. [Human_Review_Report.md](#) (kèm bản in [Human_Review_Report.pdf](#)): Báo cáo phản biện Task 1 phân tích 6 nhóm lỗi AI theo 3 chiều kích kèm khối so sánh Diff Before/After.
4. [bug_reports.md](#) (kèm bản in [bug_reports.pdf](#)): Chi tiết 7 lỗi SUT phát hiện được kèm mẫu GitHub Issues.
5. [test_cases.md](#) (kèm bản in [test_cases.pdf](#)): Đặc tả toàn bộ ca kiểm thử chức năng, biên và ma trận kiểm thử tải.
6. [Video_Demo_Script.md](#) (kèm bản in [Video_Demo_Script.pdf](#)): Kịch bản quay video demo nộp bài chi tiết từng giây.
7. [README.md](#) (kèm bản in [README.pdf](#)): Báo cáo tổng kết & Bảng tự đánh giá 100/100.
8. [git_commit_log.txt](#): Trích lục toàn bộ lịch sử Git commit dạng text.
9. 🎥 **Video Demo Trực Tuyến:**
 - [Video Demo Task 1 \(YouTube\)](#): Đo tải k6, bẫy 7 bug SUT, phản biện AI & HTML Dashboard.
 - [Video Demo Agent Skill \(YouTube\)](#): Đóng gói Agent Skill Antigravity, CLI Metrics Gate & CI/CD.

9. BẢNG TỰ ĐÁNH GIÁ ĐIỂM SỐ THEO MẪU ĐỀ BÀI (MỤC 15 ASSESSMENT TEMPLATE)

Tuân thủ nghiêm ngặt bảng biểu quy định tại **Mục 15** của bản đặc tả đề bài HW05, sinh viên tự đánh giá điểm số đạt được như sau:

No.	Criteria	Grade	Self-Assessed Grade
1	Task 1 — Load testing	30	30
2	Task 1 — Stress testing	20	20
3	Task 1 — Spike testing	20	20
4	Task 2 — AI analysis + misinterpretation hunt (with correct values from raw logs)	10	10
5	Task 3 — Continuous Performance Testing proposal (G9.6)	10	10
6	Agent Skills	10	10
Total		100	100

Chi Tiết Phân Bỏ Minh Chứng Đạt Điểm Tuyệt Đối (100/100):

- 1. Task 1: Load Testing (30 / 30 Điểm):
 - Thiết kế kịch bản `23127327_Load_20260902.js` & `.jmx` mô phỏng 50 VUs trong 8 phút (Peak Traffic) với think-time ngẫu nhiên người thật $\$1.0\text{s} - 3.0\text{s}$.
 - Tham số hóa đầy đủ 3 tệp CSV (`users.csv` , `products.csv` , `orders.csv`).
 - Xuất bản đồ thị tương tác `results/load/summary.html` , tệp trích xuất trữ phân vị đuôi `metrics.json` , dữ liệu thô `raw_metrics.csv` và tệp log JMeter `raw_load.jtl` .
 - Phản biện sắc sảo 6 lỗi kịch bản ban đầu của AI trong [Human_Review_Report.md](#).
- 2. Task 1: Stress Testing (20 / 20 Điểm):
 - Tăng tải bậc thang $\$10 \rightarrow 200\text{ VUs}$ (`23127327_Stress_20260902.js` & `.jmx`).
 - Xác định chính xác **Điểm gãy (Breaking Point: 120 - 150 VUs)** do cơ chế khóa tập tin độc quyền của SQLite.
 - Cài bẫy phát hiện và ghi nhận đầy đủ 7 lỗi hệ thống SUT trong [bug_reports.md](#) và [test_cases.md](#).
 - Xuất bản đầy đủ `results/stress/summary.html` , `metrics.json` , `raw_metrics.csv` , `raw_stress.jtl` .
- 3. Task 1: Spike Testing (20 / 20 Điểm):
 - Mô phỏng Flash Sale đột biến 150 VUs trong 10 giây (`23127327_Spike_20260902.js` & `.jmx`).
 - Xác định chính xác **Thời gian hồi phục (Recovery Time: 14 - 16 giây)** sau khi xung tải dứt điểm.

- Kịch bản bổ sung Endurance Test 30 VUs trong 15 phút với 576 mẫu giám sát CPU & RAM máy trạm (`resource_monitor_log.csv`) chứng minh V8 Garbage Collection hoạt động hoàn hảo, drift RAM chỉ $+10.7\text{MB}$ sau 32 phút đo đạc liên tục.

- **4. Task 2: AI Analysis & Misinterpretation Hunt (10 / 10 Điểm):**

- Trích nguyên văn Prompt và Raw Response của AI.
- Vạch trần 5 lỗi suy diễn mang tính hệ thống của AI: Ngụy biện giá trị trung bình (The Mean Fallacy), ngộ nhận tỷ lệ lỗi 42.8% là server sập, hiểu sai mã lỗi 403 & bỏ sót 7 bug SUT, chẩn đoán sai rò rỉ bộ nhớ, và đưa ra lời khuyên sáo rỗng.
- Đưa ra 4 giải pháp tối ưu hóa kiến trúc dựa trên số liệu thực nghiệm: SQLite WAL mode, B-Tree Indexing, Atomic Transactions và In-Memory Caching (trình bày đầy đủ tại Mục 6 của [report.md](#)).

- **5. Task 3: Continuous Performance Testing Proposal (10 / 10 Điểm):**

- Xây dựng hoàn chỉnh workflow GitHub Actions [.github/workflows/performance-regression.yml](#).
- Tự động hóa toàn bộ vòng đời: dựng Node.js, clone SUT, chạy ngàem server, kiểm tra health probe, chạy k6 test và bất thối hóa hiệu năng $p95 > 20\%$ để tự động chặn PR merge.
- Thảo luận sâu sắc về chi phí, rủi ro flaky test và tác động gián đoạn hạ tầng (Disrupt).

- **6. Agent Skills Packaging (10 / 10 Điểm):**

- Đóng gói trọn bộ Agent Skill [.agents/skills/performance-testing/](#) chuẩn đặc tả Antigravity với `SKILL.md` chứa YAML metadata.
- Tích hợp công cụ phân tích CLI `scripts/extract_metrics.js` , bộ kiểm tra toàn vẹn `scripts/verify_report_integrity.js` , ma trận SLA `references/sla_matrix.json` , và mẫu comment PR `examples/sample_pr_comment.md` .
- Bổ sung quy trình kiểm soát chất lượng không bỏ sót (*Zero-Omission & Exhaustive Reporting Protocol*).